LEVERAGING MACHINE LEARNING TO PREDICT FOOD ITEM SALES

GITHUB: https://github.com/Ouambo-SIM/Sales-Prediction-with-ML-and-M5-data.git

DECEMBER 10, 2025

DATA SCIENCE INSTITUTE
AUTHOR: FRANCK IVAN OUAMBO SIMO

Introduction

An efficient supply chain often requires the right material replenishment strategies. However, developing such strategies can necessitate years of product and sales experience, which is difficult to acquire. Moreover, most products exhibit highly volatile sales figures, and in such cases, even years of experience may not be sufficient.

Machine learning models can help democratize advanced supply chain optimization methods, as they can identify intricate patterns invisible to the human eye, even in a complicated case like sales prediction.

The dataset used for this project consists of Walmart sales data previously provided for the M5 competition, where numerous researchers tested a wide range of methods to predict sales at specific dates in the future.

For this project, sales data for one product at a Texas Walmart store was used, with the goal of predicting its future sales using prior sales data.

The dataset comprises Walmart sales data from January 29, 2011, to June 19, 2016 (approximately 5.4 years). The table below describes the dataset's content.

Data category	Elements
Temporal data	Day, week, month, Date
Event data	Sports, cultural, and religious events
Pricing data	Product price
Sales data	Quantity sold
Other	Store, state, department and Snap payment eligibility (binary values)

Previous attempts at predicting future sales on this dataset can be examined in the M5 competition GitHub Repository here. During the competition, multiple machine learning models were tested including:

-Multi-Layer Perceptron

-Random Forest

-Global Multi-Layer Perceptron

-Global Random Forest

Among the competition's goals, the one most similar to mine was 28-day-ahead point forecasting. The models after training and validation were then tested on 28 days of test data previously unavailable.

Many more models were used by participants but for comparison with my project's results, the competition used Weighted Root Mean Squared Scaled error (WRMSSE) as evaluation metric and the average error across the machine learning models was 0.949, with Global Multi-Layer Perceptron having the lowest error of 0.882.

EDA

The following EDA shows some of the most interesting, unexpected, and important insights.

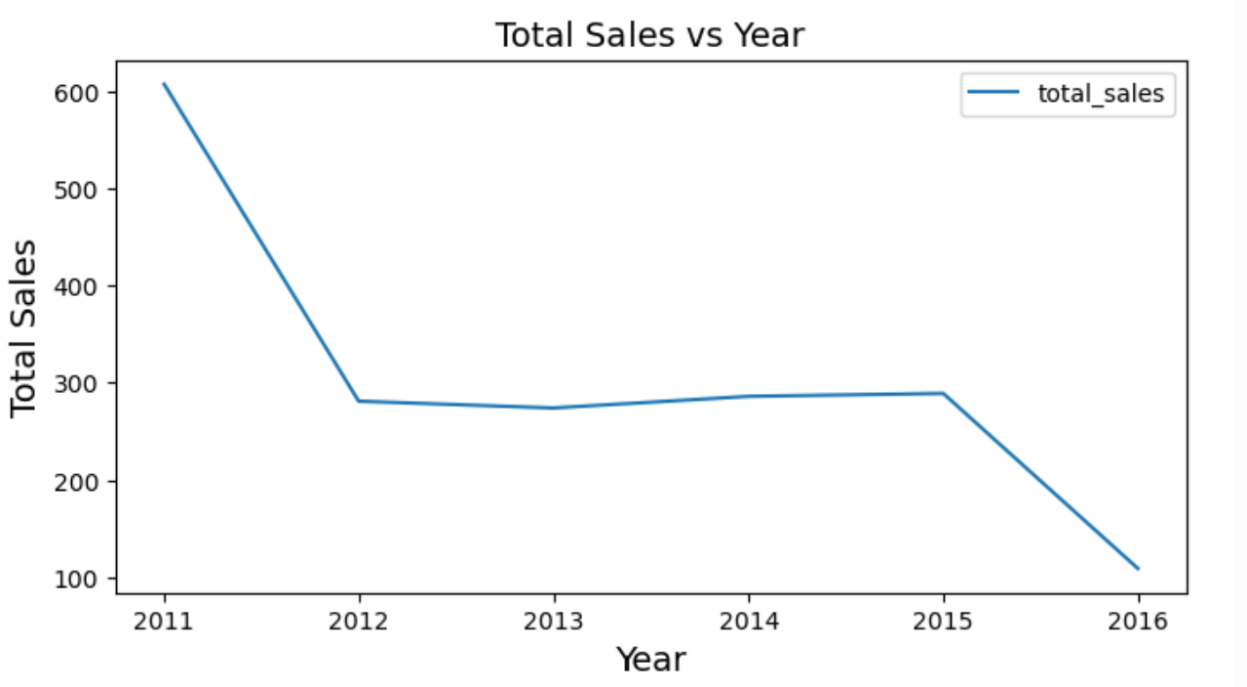

Figure: Total sales over the years from 2011 to 2016

The graph above shows that sales had been decreasing from 2011 to 2012, and stayed about the same from 2012 to 2015 after which it dropped. This is very important because it indicates that using past sales data too far in the past might mislead the model if key features like year and month are not used in training. Also, since this is a time series problem, the temporal order of data points matters. Hence, there might not be enough 2016 data available for training and testing.

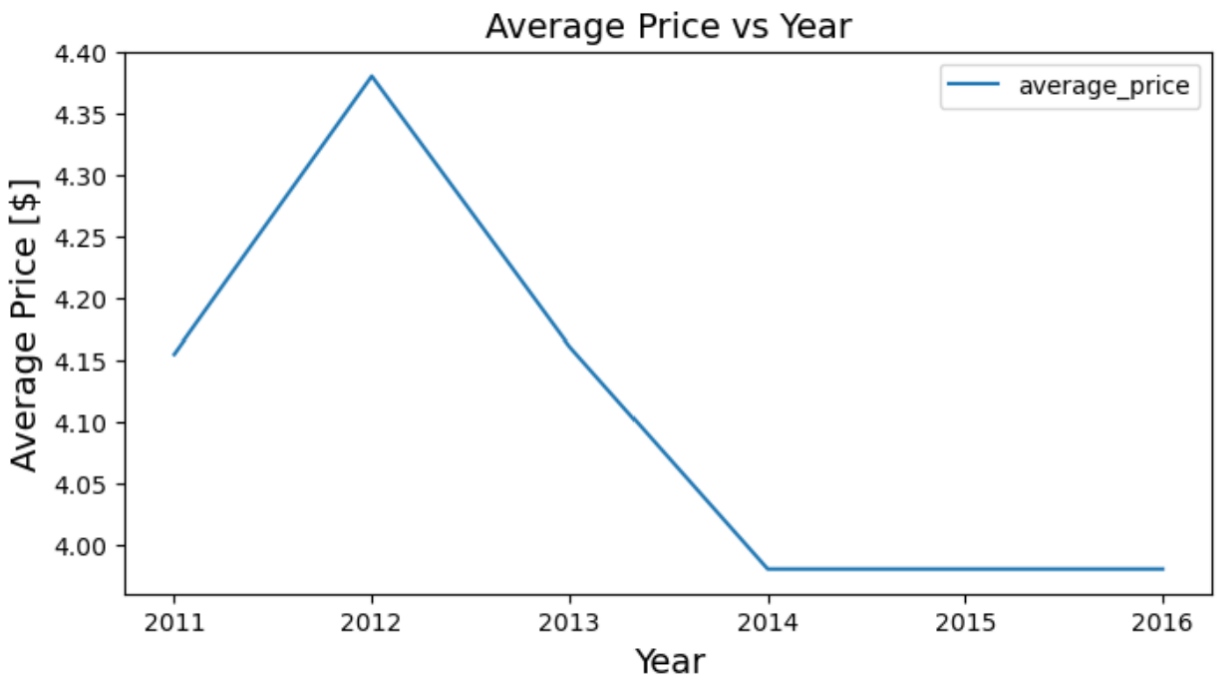

Figure: Average product price over the years from 2011 to 2016

This is an interesting graph when couples with the previous one. It indicates that after experiencing high sales in 2011, Walmart increased the specific product's prices to make more profit in the subsequent years. However, it experienced considerably lower sales instead. In response, the store kept reducing its prices to increase sales and plateaued somewhere below an average price of $4 which was probably Walmart's price limit.

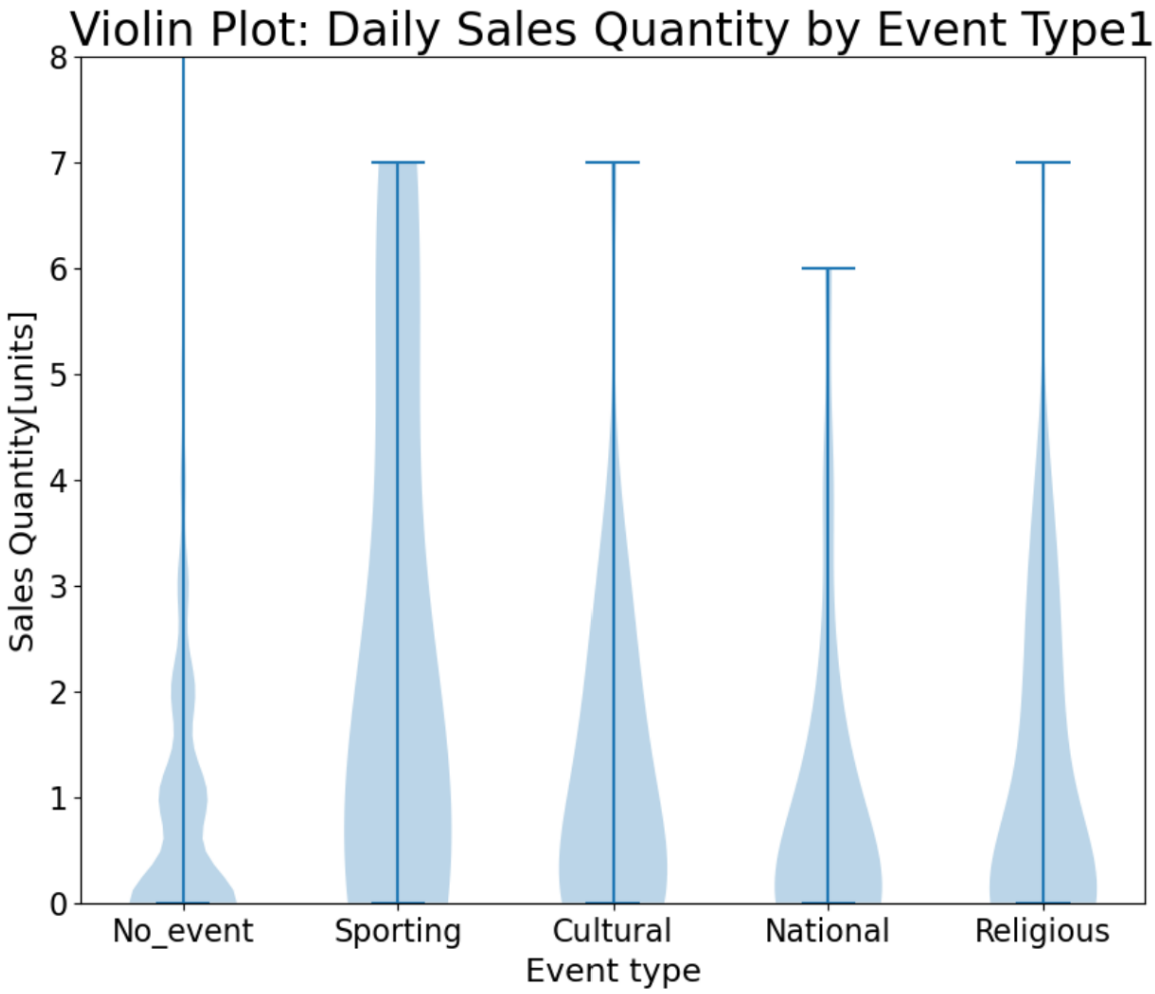

Figure: Violin plot demonstrating sales volumes for each event type at various Quantities

Note: There is another field called EventType2, signifying the second event Type happening on a certain day. However, the field Event Type1 largely has more values comparatively.

The Violin plot above shows that sales volumes are considerably increased during events, with sporting events yielding the largest volumes overall.

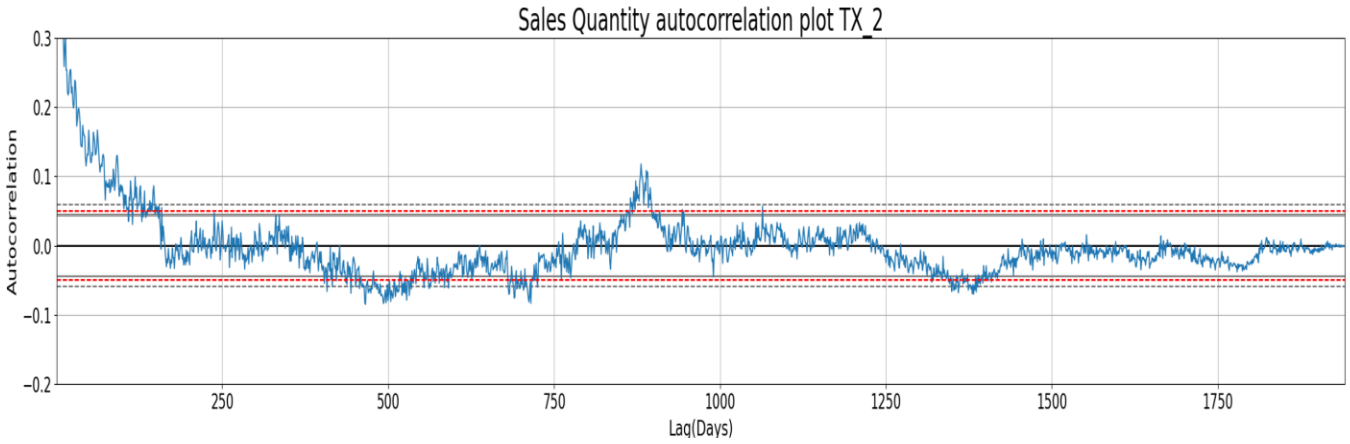

Figure: Autocorrelation plot of Sales quantity demonstrating correlation between current sales and past sales as Lags increase.

The autocorrelation curve above gets into the noise region around 180 lags. Low Model test scores(RMSE) on data with over 180 lags will be strongly indicative of bugs in my code or, even data leakage.

Methods

Before any processing, ID fields and duplicate columns were excluded, while the day part of the 'date' feature was extracted and added to the dataset

Then, the dataset was modified to include vector autoregression features. For every data point, vector autoregression features were added to reflect SNAP eligibility, events, prices, temporal elements (month, day, year) and sales on days prior to that of the prediction. Also, each data point's initial features were preserved, except for Sales Quantity to prevent data leakage. For prediction, this allowed the model to identify relationships between events , Snap, price, position in time and sales quantity.

The Number of Lags is a critical hyperparameter tuned separately. Several lag values on an exponential scale are tested to find the lag that yields the minimum Root Mean Squared Error, the cost function. Early stopping was implemented here to stop lagging when the model continuously performs worse or does not significantly improve as lags increase a certain number of times (threshold= <0.2% improvement). Within each iteration of lag values, the following happens.

Various models are trained on the dataset:

- Lasso Regression
- Ridge Regression
- Random Forest
- XGBoost

The models are fed into an ML Pipeline, along with the initialized preprocessor, vector regressed data set, corresponding parameter grid and an integer for the number of splits.

My Dataset is a time series one with sales quantity as the target variable. This implies that temporal order needs to be maintained to prevent data leakage. As such, in the next step, I use sklearn's train_test_split without shuffling, and TimeSeriesSplit(in that order) as splitting strategy. Train_test_split separates the last 20% of my data set(388 data points) for use as test set. Then, TimeSeriesSplit, creates five training/validation set splits, which are later used for finding the best hyperparameters for my models.

Next, the dataset is preprocessed. At this point, the vector autoregression features increased the number of columns hence, new feature name lists are needed to define the preprocessor. For this, I use a function which uses category lists of feature names along with the number of lags as input to create an extended list of the original feature categories to reflect the increased number of features from lagging. Then, with the feature lists, the preprocessor is initialized with sklearn's ColumnTransformer.

Other than sklearn's encoders, I used a custom one called cyclical encoder to change the month and weekday columns into two columns each: sine and cosine. That encoder passes columns through a sine and cosine function to create new features that depict month and weekday cycles while allowing the model to still pinpoint a difference in values at close points of a cycle. The table below depicts encoders used

Feature Types	Encoder
Month	Cyclical Encoder
Weekday	Cyclical Encoder
Categorical	Onehot
Ordinal	Ordinal
Continuous features	Standard Scaler

An sklearn GridSearchCV function is then initialized with a Pipeiline object consisting of the preprocessor and model to use, a scoring parameter of 'neg_root_mean_squared_error' , with cv as TimeSeriesSplit with 5 splits. This is fit on the dataset(without the test set). Then, the resulting best estimator is used to make predictions on the test set. With that, a test score(Root mean Squared error) is evaluated and returned along with the fitted best estimator.

For XGboost specifically, I made a GridSearchcv function that takes it as model input, its associated parameter grid, preprocessor , splitting method, the features and target variable and mean_squared error as scoring method. Here, all parameter combinations in the parameter grid are iterated through. Within each combination, for each split obtained from the splitting method(TimeSeriesSplit), specific row indices in the train/val set are used as train set or validation set. The XGBoost model is trained on the training set and error scores are evaluated from its prediction on the validation set. The parameter combination that generated the lowest score is then returned, used to initialize a pipeline fit on the train/val set, and used to get the RMSE from predictions on the test set .

The best Root Mean square error(RMSE) obtained for all the models are compared to determine the best model for the project's use case. RMSE was used as evaluation metric because it is an easily interpretable measure of how good each model is and has the same unit as the target variable. RMSE tells me how far the models' predictions were from the true target variable values.

The hyperparameters used for each model are summarized below:

Model	Hyper Parameters
Lasso Regression	alpha: np.logspace(-3, 3, 7) max_iter: 10000
Ridge Regression	Alpha: np.logspace(-3, 3, 7) solver: saga max_iter: 10000
Random Forest	Max_depth: [2,20] random_state: [42, 84, 126, 168]
XGBoost	'n_estimators': [10000] 'alpha': [0e0, 1e-2, 1e-1, 1e0, 1e1, 1e2] 'lambda': [0e0, 1e-2, 1e-1, 1e0, 1e1, 1e2] 'max_depth': [1,3,10,30] 'seed': [0] 'learning_rate': [0.03] 'subsample' : [0.66] 'colsample_bytree': [0.9] 'missing': [np.nan]

Alpha is the regularization parameter which reduces the weights assigned to each feature's values, hence preventing overfitting and enhances model generalization. In Lasso Regression, alpha is the L1 regularization parameter while in Ridge, it is the L2 regularization parameter.

Maximum iteration is the number of tries the model has to converge and is set to 10000 overall.

The saga solver is used in ridge regression. Instead of calculating the full gradient of the loss function over the entire dataset in each iteration (which is slow for large datasets), SAGA estimates the gradient using a single sample (or a small batch) at a time.

Max_depth parameter for Random forest and XGBoost prevents trees from overfitting.

Random_states in random forest finds the seed that yields the best cost function value.

Lambda in XGBoost is the L2 regularization parameter while alpha is the L1 regularization parameter

Learning rate determines the size of the step taken in modifying weights for the prediction function.

Subsample and colsample_bytree determine the fraction of datapoints and features used in building the trees in XGBoost, hence improving model generalization.

The “missing” parameter in XGboost tells the model what values should be treated as missing in the data.

The following table shows the effect of the splitting strategy, and nondeterministic ML models on the evaluation metric.

models	random state effect(stdev)	split strategy effect(stdev)
Linear regression; L1 reg	0	0
Linear regression; L2 reg	0	0
Random Forest	0.004	0.005
XGBoost	0	0.009

Note: table data obtained after iterating through random states: [10,30, 45, 50], and timeseries split with n_splits: [3, 4, 5, 6]

Results

The baseline score is 1.436 units. The following table describes my models’ RMSE in relation to the baseline score.

Models	Test score(RMSE)	Distance from Baseline(STDEV)
Linear regression; L1 reg	0.97	0.466
Linear regression; L2 reg	1.032	0.404
Random Forest	0.979	0.457
XGBoost	1.109	0.327

The most predictive models were Linear regression with L1 regularization and Random Forest with the former achieving the lowest score overall of 0.97 Units

The following figures demonstrate global importance measurements of all the features, with sales quantity one day prior seeming to be the most predictive feature.

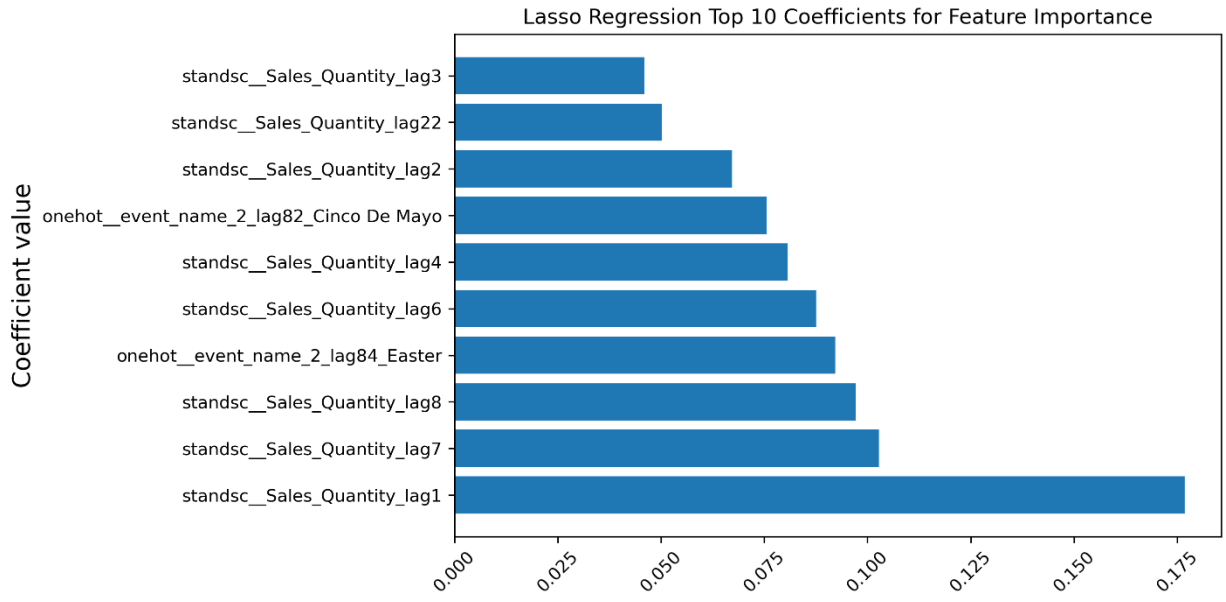

Figure: Coefficients of Top 10 most important features in the most predictive model Lasso Regression

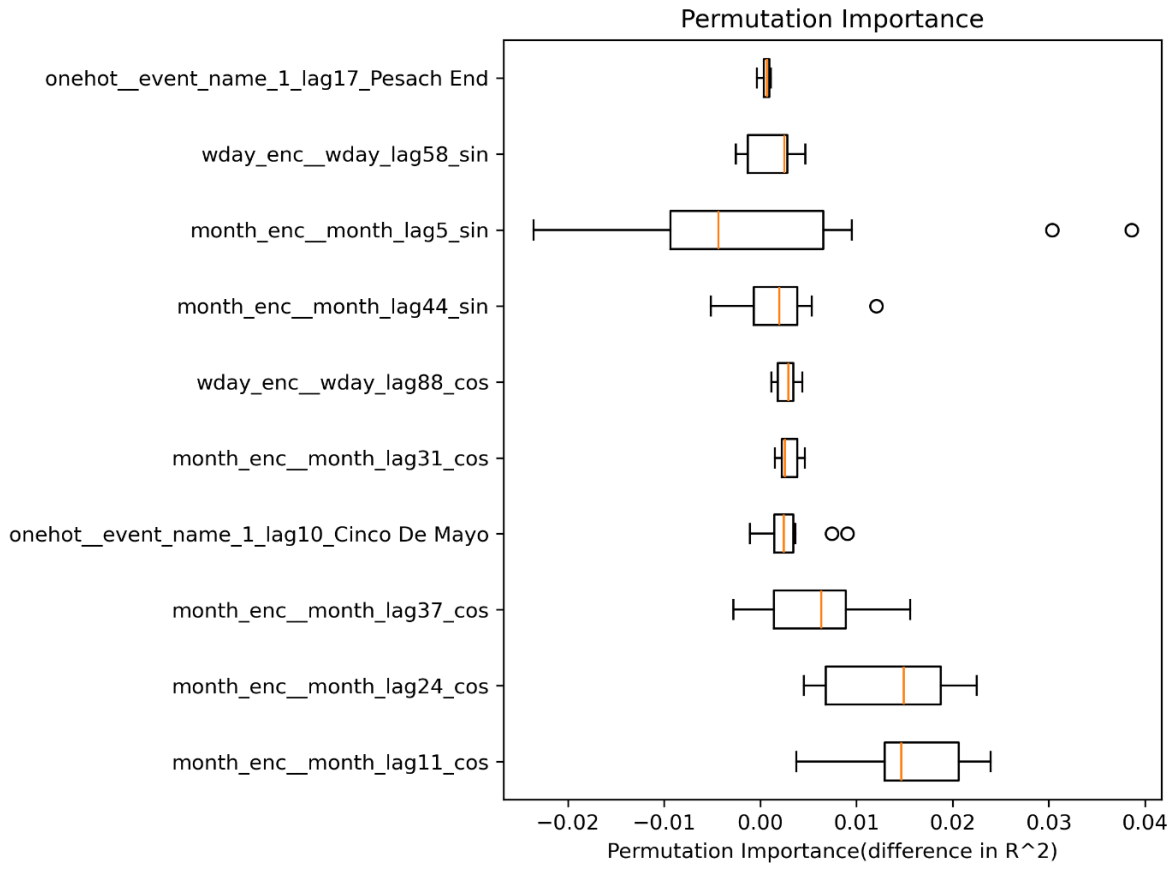

Figure: Top 10 features in the most predictive model. Evaluation was made using permutation importance method

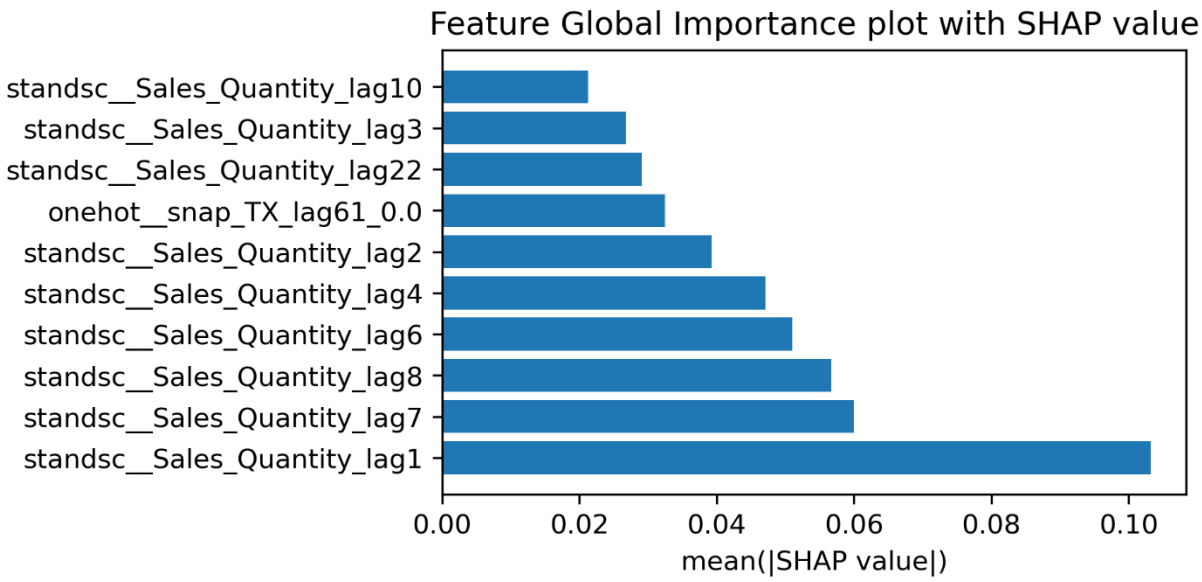

Figure: Top 10 features in the most predictive model. Evaluation was made using SHAP

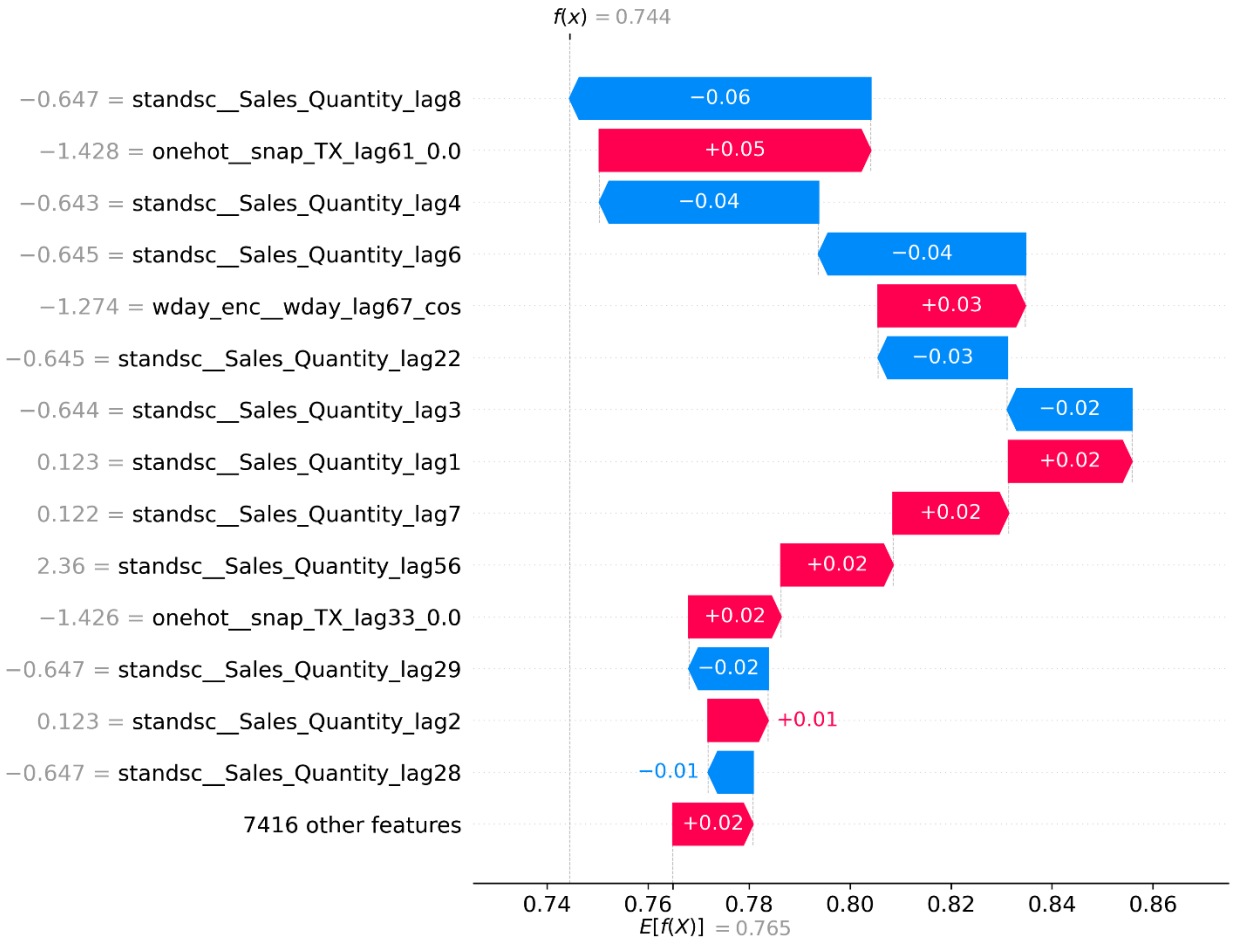

Figure: Most predictive features in the 99th data point in dataset.

From the model interpretation plots for both local and global feature importance, generally, one can say for a specific day, using the most predictive model in this project, sales within the 7 days prior and about a month prior(22 days earlier) are very important in the model's predictions.

On the other hand, the least important features are many. Several features seem to barely affect the target variable, if at all. Among the least important features, the ones that provide some sort of effect are event_name_1_lag_50_NBAFinalsStart, Sales quantity_lag56(over a month ago), and Sales_Quantity_lag14(two weeks prior).

What's surprising is the sheer number of features that did not provide any effect or barely any in making predictions. During tuning, the Lasso regression model required 128 lags for most features in order to reach the minimum RMSE . However, when interpreting the models no feature anywhere near the 128 lag mark significantly impacted predictions.

Outlook

The weak spots in my modeling approach are mainly the elements my best model is based on. Looking at the models I tested, the Linear regression models with L1 and L2 regularization look more like a VAR model (due to the autoregression features). AR models assume a stationary dataset, which is not the case in my dataset due to a changing sales mean over time. To make my models more predictive, adding other features such as prediction error at each data point, replacing the lagged sales quantities by the lagged change in sales, and predicting the change in sales instead would be a great approach. The result would then be the sum of previous sales and predicted change in sales. Those additional features would provide even more features my models can use to make better predictions.

Additional Data that could improve my model is the location of the product in the store, and even a binary feature telling whether the product was being advertised on each date. These are factors that often affect sales as hidden products or products not well known often have lower sales.

References

Kaggle. (2020). *M5 Forecasting – Accuracy: Competition data*. Retrieved from https://www.kaggle.com/competitions/m5-forecasting-accuracy/data

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020). *The M5 competition: Methods and results*. GitHub repository. Retrieved from https://github.com/Mcompetitions/M5-methods